

Reference Open Source HESIA CoreLib FR

****Projet**:** HESIA CoreLib

****Version**:** 0.1.0

****Langue**:** Francais

****Perimetre**:** bibliotheque open source de securite et de controle d'inference pour systemes embarques

****Audience**:** ingenieurs firmware, ingenieurs securite, mainteneurs OP-TEE, ingenieurs pipelines IA, mainteneurs de bindings

1. Objectif

HESIA CoreLib est la bibliotheque open source qui donne au projet HESIA une surface technique publique, auditable et reutilisable sans exposer le runtime produit prive.

L'objectif est comparable dans l'esprit a OpenSSL: fournir une base petite, stable et maintenable sur laquelle d'autres projets peuvent construire. CoreLib n'est pas un clone d'OpenSSL et n'implemente pas de primitives cryptographiques maison. La bibliotheque delegue les operations cryptographiques a des fournisseurs reconnus et se concentre sur l'integration, la stabilite ABI, les erreurs, le comportement fail-closed, la maintenabilite et les controles de securite autour des pipelines IA embarques.

CoreLib permet a des ingenieurs externes d'evaluer HESIA par du code reel:

- ABI C stable
- implementation lisible
- cryptographie par fournisseurs
- modele d'erreur explicite
- comportement fail-closed
- watchdog et garde d'inference
- bindings C, C++, Python, Rust, Swift et Ada

2. Separation public / prive

Le produit HESIA reste partiellement ferme parce qu'il contient du firmware specifique, du durcissement operationnel, des flux OP-TEE, de l'integration modele et du code client sensible.

La bibliotheque open source reste volontairement limitee aux primitives reutilisables:

- gestion commune des erreurs
- zeroisation securisee
- octets aleatoires
- surface fournisseur AES-256-GCM
- surface fournisseur ML-KEM et ML-DSA

- machine d'etat watchdog
- validation de requete d'inference
- ABI multi-langage

Le produit ferme peut utiliser CoreLib en interne, mais CoreLib doit rester utile seule.

3. Principes de conception

3.1 ABI stable d'abord

L'interface primaire est en C. Tous les autres bindings appellent la meme ABI C. Cela reduit le cout de maintenance et evite d'avoir cinq comportements differents entre Python, Rust, Swift, C++ et Ada.

3.2 Pas de cryptographie maison

CoreLib n'invente pas de chiffrement symetrique, de signature, d'echange de cle ou de KDF. L'API publique peut exposer des operations post-quantiques, mais l'implementation est deleguee a OpenSSL et liboqs.

3.3 Fail-closed

Si un fournisseur n'est pas compile, l'API ne degrade pas silencieusement. Elle retourne HESIA_STATUS_UNSUPPORTED. Un logiciel de production doit traiter une fonctionnalite obligatoire non supportee comme un echec de deploiement.

3.4 Maintenance plutot que cleverness

Le code privilegie les API plates, les tailles explicites, les buffers appartenant a l'appelant et les codes statut. Cela rend la bibliotheque plus simple a binder, a auditer et a maintenir.

3.5 L'IA embarquee est une frontiere de securite

Les entrees, sorties, deadlines et identites de modeles sont traitees comme des donnees controlees par politique. CoreLib n'execute pas le modele. Elle valide le contrat autour de l'execution.

4. Organisation du depot

CoreLib vit dans:

`open_source/hesia-core/`

Fichiers importants:

`include/hesia/core.h`
`include/hesia/core.hpp`
`src/hesia_core.c`

ABI C stable
 Wrapper C++ header-only
 Implementation C portable

tests/test_core.c	Tests unitaires
bindings/python/hesia_core.py	Binding Python ctypes
bindings/rust/	Crate Rust FFI
bindings/swift/	Surface Swift Package
bindings/ada/hesia_core.ads	Spec Ada
examples/c/quickstart.c	Exemple C
examples/python/quickstart.py	Exemple Python

Documentation projet:

docs/HESIA_CORELIB_EN.md
docs/HESIA_CORELIB_FR.md
docs/HESIA_CORELIB_EN.pdf
docs/HESIA_CORELIB_FR.pdf

5. Matrice fonctionnelle

Fonction	Etat API	Fournisseur	Comportement par défaut
Modele d'erreur	Implemente	Interne	Disponible
Zeroisation securisee	Implemente	Interne	Disponible
Aleatoire OS	Implemente	OS / OpenSSL	Disponible si l'OS le permet
AES-256-GCM	API implementee	OpenSSL	UNSUPPORTED sans OpenSSL
ML-KEM	API implementee	liboqs	UNSUPPORTED sans liboqs
ML-DSA	API implementee	liboqs	UNSUPPORTED sans liboqs
Watchdog	Implemente	Interne	Disponible
Garde d'inference	Implemente	Interne	Disponible
Binding C++	Implemente	Wrapper ABI	Disponible
Binding Python	Implemente	ctypes	Disponible avec bibliotheque partagee
Binding Rust	Implemente	FFI	Disponible avec bibliotheque liee
Binding Swift	Implemente	Swift Package	Disponible avec bibliotheque liee
Binding Ada	Implemente	GNAT / ABI C	Disponible avec bibliotheque liee

6. Vue d'ensemble de l'ABI C

L'ABI C est declaree dans:

open_source/hesia-core/include/hesia/core.h

Elle utilise:

- types entiers fixes de stdint.h
- size_t pour les longueurs de buffers
- buffers appartenant a l'appelant
- longueurs de sortie explicites
- hesia_status_t pour toutes les operations faillibles
- hesia_error_t optionnel pour le diagnostic humain

Exemple:

```
hesia_error_t error;
uint8_t out[32];

hesia_error_init(&error);
if (hesia_random(out, sizeof(out), &error) != HESIA_STATUS_OK) {
    fprintf(stderr, "random failed: %s\n", hesia_error_message(&error));
}
```

7. Modele d'erreur

L'ABI C ne lance pas d'exceptions. Les appels faillibles retournent un hesia_status_t.

Statuts importants:

- HESIA_STATUS_OK
- HESIA_STATUS_INVALID_ARGUMENT
- HESIA_STATUS_BUFFER_TOO_SMALL
- HESIA_STATUS_UNSUPPORTED
- HESIA_STATUS_CRYPTO_ERROR
- HESIA_STATUS_AUTHENTICATION_FAILED
- HESIA_STATUS_POLICY_DENIED
- HESIA_STATUS_WATCHDOG_EXPIRED
- HESIA_STATUS_INFERENCE_REJECTED
- HESIA_STATUS_IO_ERROR
- HESIA_STATUS_INTERNAL_ERROR

Regle pour le code de production:

Si une fonctionnalite est obligatoire et que CoreLib retourne HESIA_STATUS_UNSUPPORTED, il faut arreter l

8. Modele cryptographique

8.1 Chiffrement symetrique

L'API AEAD expose actuellement:

- HESIA_AEAD_AES_256_GCM

L'implementation est compilee uniquement lorsque le support OpenSSL est active. Sans OpenSSL, l'API retourne HESIA_STATUS_UNSUPPORTED.

8.2 Algorithmes post-quantiques

L'API PQC expose actuellement:

- HESIA_PQC_ML_KEM_768
- HESIA_PQC_ML_KEM_1024
- HESIA_PQC_ML_DSA_65
- HESIA_PQC_ML_DSA_87

L'implementation est compilee uniquement lorsque le support liboqs est active. Sans liboqs, l'API retourne HESIA_STATUS_UNSUPPORTED.

8.3 Regle fournisseur

CoreLib possede:

- la stabilite d'API
- les controles de taille
- le comportement fail-closed
- la coherence des bindings

Les fournisseurs possedent:

- l'implementation des primitives cryptographiques
- les proprietes bas niveau propres aux algorithmes
- la generation de cle, l'encapsulation, la signature et la verification

9. Module watchdog

Le watchdog est une petite machine d'etat pour boucles embarquees et pipelines IA:

1. initialiser avec un timeout et un nombre de deadlines manquees autorise
2. armer
3. envoyer un heartbeat depuis la boucle controlee
4. verifier depuis un superviseur
5. echouer avec HESIA_STATUS_WATCHDOG_EXPIRED si la politique est violee

Cas d'usage:

- boucles d'inference
- ingestion capteurs
- workers de canal securise
- outils de maintenance OP-TEE
- etapes de pipeline ou un blocage silencieux est dangereux

10. Module garde d'inference

CoreLib n'exécute pas de réseau neuronal. Elle valide le contrat d'exécution avant l'appel au moteur modèle.

Le contrat contient:

- identifiant du modèle
- version du modèle
- taille d'entrée maximale
- taille de sortie maximale
- politique de latence maximale
- exigence optionnelle de digest pour modèle signé

La requête contient:

- pointeur et longueur d'entrée
- buffer et capacité de sortie
- deadline demandée

La garde rejette:

- identité modèle absente
- limites non configurées
- entrée trop grande
- sortie trop grande
- deadline hors politique
- digest absent quand la politique exige un modèle signé

Cela rend les pipelines IA plus auditable car la frontière de sécurité est placée avant le moteur d'inference.

11. Bindings langage

11.1 C

C est l'ABI primaire et la cible la plus simple pour firmware, outils host OP-TEE et systemes embarques contraints.

11.2 C++

Le header C++ transforme les codes statut en exceptions et fournit une aide RAI pour le watchdog.

11.3 Python

Le binding Python utilise ctypes. Il vise les tests, l'outillage operations, les probes CI et les scripts d'integration.

11.4 Rust

Le crate Rust expose les definitions FFI et quelques wrappers sur version, features, random bytes et watchdog.

11.5 Swift

Le Swift Package encapsule la cible C et expose une petite surface d'erreur native Swift.

11.6 Ada

La spec Ada mappe l'ABI C pour GNAT et les bases de code orientees surete.

12. Recettes de build

12.1 Build portable minimal

```
cd open_source/hesia-core
cc -Iinclude -c src/hesia_core.c -o hesia_core.o
ar rcs libhesia_core.a hesia_core.o
cc -Iinclude tests/test_core.c libhesia_core.a -o test_core
./test_core
```

Sur Windows avec GCC, lier bcrypt pour l'aleatoire OS:

```
gcc -Iinclude tests/test_core.c src/hesia_core.c -lbcrypt -o test_core.exe
test_core.exe
```

12.2 Build CMake

```
cmake -S open_source/hesia-core -B open_source/hesia-core/build
cmake --build open_source/hesia-core/build
ctest --test-dir open_source/hesia-core/build
```

12.3 Build avec fournisseurs

```
cmake -S open_source/hesia-core -B open_source/hesia-core/build-provider \
-DHESIA_CORE_WITH_OPENSSL=ON \
```

```
-DHESIA_CORE_WITH_LIBOQS=ON  
cmake --build open_source/hesia-core/build-provider
```

Pour liboqs, definir LIBOQS_ROOT si le fournisseur n'est pas installe dans un chemin standard.

13. Pattern d'integration OP-TEE

CoreLib ne remplace pas la Trusted Application OP-TEE. La bonne separation est:

- la TA OP-TEE possede les secrets scelles, les cle de signature, la meta rollback et la politique liee au materiel
- CoreLib possede l'ABI cote host, les controles de politique, les erreurs et l'integration langage

Integration recommandee:

1. Utiliser les erreurs et statuts CoreLib dans les outils host.
2. Utiliser les helpers de taille PQC et de verification hors TA.
3. Garder la signature privree ML-DSA dans la TA ou le HSM en production.
4. Traiter toute signature normale-world comme non-production.
5. Refuser le demarrage si les commandes TA requises sont indisponibles.

14. Exigences securite pour une release

Avant d'appeler une release CoreLib prete production:

- builder et tester chaque architecture cible
- figer les versions OpenSSL et liboqs
- executer des controles de compatibilite ABI
- fuzzing sur les API buffers, AEAD et garde d'inference
- tests negatifs sur les fournisseurs absents
- verification du fail startup pour fonctionnalites obligatoires
- validation de tous les bindings contre la meme bibliotheque partagee
- publication SBOM et licences
- revue externe avant usage haute assurance

15. Modele de maintenance

CoreLib doit garder une politique de compatibilite conservatrice:

- patch: correction sans changement ABI

- mineur: ajout de fonctions ou valeurs enum
- majeur: rupture ABI uniquement avec guide de migration
- API depreciees conservees au moins un cycle mineur

Regle stable:

Ne pas rendre les bindings langage plus intelligents que l'ABI C.

16. Roadmap

Court terme:

- CI Windows, Linux et ARM64
- validation provider contre l'arbre liboqs vendore de HESIA
- script de controle ABI
- fuzzing AEAD et garde d'inference
- exports pkg-config et CMake package
- packaging Python wheel

Moyen terme:

- adaptateurs outils host OP-TEE
- evenements d'audit structures
- verificateur de manifestes modeles signes
- vecteurs de test pour builds avec fournisseurs
- exemples d'integration pour pipelines IA drone

Long terme:

- revue securite externe
- signature des releases
- archives source reproductibles
- engagement public de stabilite API

17. Validation realisee

Validation collectee le 2026-05-01:

- Build local Windows GCC de tests/test_core.c avec src/hesia_core.c: reussi.
- Build et execution locale Windows du quickstart C: reussi.
- Build local Windows de la bibliotheque partagee chargee par le binding Python ctypes: reussi.
- Compilation et execution locale Windows du wrapper C++ via include/hesia/core.hpp: reussi.

- Compilation locale Windows de la spec Ada avec GNAT: reussi.
- Build et execution C distante sur Jetson Orin Nano Super ajax@100.101.152.53: reussi dans /tmp/hesia-core-codex-20260501111412.

Cette validation couvre le socle sans fournisseurs: erreurs, zeroisation, watchdog, garde d'inference, disponibilite aleatoire OS et comportement fail-closed lorsque OpenSSL/liboqs ne sont pas compiles.

Les builds avec fournisseurs OpenSSL/liboqs doivent encore etre valides avec les versions exactes des fournisseurs avant toute revendication production.

18. Positionnement

HESIA CoreLib est la surface de preuve open source de la plateforme HESIA:

- native post-quantique
- maintenable
- portable entre langages
- adaptee a l'embarque
- compatible OP-TEE
- consciente des pipelines IA

Sa credibilite vient de sa retenue: la bibliotheque expose des primitives utiles, documente ses limites, delegue la cryptographie a des fournisseurs reconnus et echoue de facon fermee lorsque le deploiement ne correspond pas a la posture de securite demandee.